COE 379L Midterm Report: Atlas Laboratory Agent

Nayyir Ahsan, UT EID: na29635

Repository: `https://github.com/nayyirahsan/coe379lfa26-midterm`

1. System Overview

The agent asks the model for one decision at a time. A trusted `LabRequest` supplies the authenticated user and any resolved project, resource and time interval. The system prompt separates these trusted fields from user text, retrieved passages and tool observations. Instructions found in retrieved text are treated as data and cannot bypass the action checks.

The decision union uses `kind` to distinguish a tool proposal, a clarification question and a final response. The loop requests decisions through `DecisionAdapter`, returns clarification without tool execution, and stops after at most ten decisions by default. Before calling a tool, the loop checks its arguments, the request fields and the reservation requirements. The loop records each allowed or denied tool proposal in the trace and sends its decision and observation back to the model. Before returning a final response, it checks the citations and completed action names against successful observations in the trace.

2. Public Benchmark Results

All six public scenarios returned the expected status: one completed reservation, four blocked requests and one clarification. In `public_benchmark.json`, the success rate and clarification accuracy are both 100%. The invalid structured response rate, unsafe action attempt rate and unnecessary refusal rate are all 0%. The average number of tool calls is 2.67 per scenario.

In `success_tensile`, Alice's 10:00–11:00 reservation for `polymer-study` follows successful policy, authorization and resource-status calls. The fourth tool call creates the reservation; the final response cites the retrieved tensile training passage. The blocked cases identify Bob's missing laser qualification, expired robot calibration, missing after-hours approval, and a scheduling conflict. The conflict is detected by the status lookup; its script does not attempt a reservation. The ambiguous-time case uses no tools. No public scenario produced an unexpected status.

The unsafe action attempt metric counts scenarios with an `action_gate_denied` observation. The public scripts do not make unsafe proposals, so the zero result does not show whether the gate blocks them. The student tests check that separately.

3. Student Tests

`tests/test_student_cases.py` adds 16 deterministic cases through `ScriptedLLM`, including parameterized inputs. Together with the 23 supplied cases, all 39 tests pass. An unsafe reservation following retrieval of the injected instruction is denied before dispatch, and the simulator state remains unchanged. Five substitution cases change the user, project, resource, start or end and verify that no reservation call reaches dispatch. Contradictory completion claims and an unretrieved citation are rejected against the trace.

Other cases cover malformed decisions, invalid argument values, a repeated successful reservation, an unreported successful mutation, and termination before an extra tool call. The tests track calls to dispatch to check that denied reservations are never executed, instead of only checking the final response. The tests found two problems. Invalid dates could crash the policy check, and a repeated reservation could reach the simulator using an earlier availability result. The loop now checks arguments, handles validation errors and blocks an action if the same tool and arguments already succeeded.

4. Implementation Decisions

The implementation reuses the supplied tool schemas, context builder, dispatcher and simulator. The request check rejects changed user, project, resource or time values. It also rejects reservations when required fields are still missing from the trusted request. Each failed reservation requirement produces an error explaining why the call was denied. Denied proposals remain visible as `action_gate_denied` observations. The termination reason records whether the run ended with clarification, a normal final response, an invalid decision, an inconsistent response or the step limit.

Final validation accepts only retrieved citation IDs and action names backed by successful mutating observations. A completed response needs a successful mutation and a completed action; a non-completed response cannot hide a successful mutation. These checks validate the structured fields, but they do not check every claim in the response text. A successful policy search also does not prove relevance or policy completeness. The scripted tests show what happens for the tested proposals. They do not prove that every possible request is safe or that a live model will behave correctly. The step limit bounds iterations only if model and tool calls return. The separate `data_flow_diagram.md` follows the concrete successful reservation through these stages.